

(Objectives)

1. Organizing Memory Hardware

کار کند (و حافظه‌ی کش RAM مثل) سیستم‌عامل باید بتواند با سخت‌افزار حافظه
روش‌های مختلفی برای سازمان‌دهی حافظه وجود دارد:
حافظه‌ی ساده (بدون تقسیم‌بندی)
(Segmentation) تقسیم‌بندی
(Paging) صفحه‌بندی
ترکیبی از آن‌ها

این سازمان‌دهی به سیستم‌عامل کمک می‌کند تا بداند هر پردازش در کدام قسمت از حافظه قرار گرفته است.

2. Memory-Management Techniques

سیستم‌عامل باید روش‌هایی برای مدیریت حافظه داشته باشد:
هر بار یک برنامه در حافظه بارگذاری می‌شود: (Single Contiguous Allocation) بارگذاری ساده
حافظه به بخش‌های مختلف تقسیم می‌شود: (Partitioning) پارتیشن‌بندی
حافظه به بلوک‌های کوچک (صفحه‌ها و قاب‌ها) تقسیم می‌شود: (Paging) صفحه‌بندی
حافظه بر اساس منطق برنامه (کد، داده‌ها، استک) تقسیم می‌شود: (Segmentation) تقسیم‌بندی
استفاده همزمان برای افزایش انعطاف و امنیت: Paging + Segmentation ترکیب

آدرس منطقی که پردازش تولید می‌کند ابتدا وارد Segmentation Unit می‌شود.

بخش انتخاب‌شده سپس به Paging Unit می‌رود.

در نهایت با کمک جدول صفحه (Page Table) به یک آدرس فیزیکی در RAM نگاشت می‌شود.

مثال ساده

فرض کن یک برنامه ساده داریم که شامل کد (Code)، داده‌ها (Data) و استک (Stack) است:

در روش Segmentation، هر کدام در یک بخش جداگانه قرار می‌گیرند.

حال اگر بخش داده (Data Segment) بزرگ باشد، سیستم آن را به صفحه‌ها تقسیم می‌کند و در مکان‌های مختلف RAM قرار می‌دهد.

این ترکیب باعث می‌شود هم انعطاف Segmentation داشته باشیم (برنامه راحت‌تر ساختار پیدا می‌کند) و هم کارایی Paging (اجازه می‌دهد بخش‌های بزرگ در هر جای RAM ذخیره شوند).

نکات کلیدی

مدیریت حافظه هدفش استفاده‌ی بهینه از RAM و افزایش کارایی سیستم است.

تکنیک‌های مهم: Segmentation، Paging، و ترکیب آن‌ها.

پردازنده‌های مدرن مثل Intel Pentium از Segmentation + Paging پشتیبانی می‌کنند.

آدرس‌دهی منطقی ابتدا توسط Segmentation و سپس توسط Paging به آدرس فیزیکی نگاشت می‌شود.

(Background)

برای اینکه یک برنامه اجرا شود، باید ابتدا از دیسک (مانند هارد یا SSD) وارد حافظه اصلی (RAM) شود. سیستم‌عامل این برنامه را داخل یک پروسه (Process) قرار می‌دهد تا CPU بتواند آن را اجرا کند.

برنامه باید در حافظه اصلی بارگذاری شود CPU نمی‌تواند مستقیماً از دیسک دستورات را اجرا کند. پس برنامه از دیسک به حافظه اصلی آورده می‌شود.

حافظه اصلی و ثبات‌ها (Registers) تنها مکان‌هایی هستند که CPU می‌تواند مستقیماً به آن‌ها دسترسی داشته باشد CPU نمی‌تواند مستقیماً به هارد یا SSD دسترسی پیدا کند.

پس هر چیزی که قرار است اجرا شود، باید به RAM و سپس به Register ها منتقل شود.

نحوه تعامل CPU با واحد حافظه (Memory Unit) CPU فقط این دو نوع درخواست به حافظه می‌دهد:

آدرس + دستور خواندن (Read Request) آدرس + داده و دستور نوشتن (Write Request) بنابراین، حافظه تنها یک جریان ساده از آدرس + عملیات خواندن/نوشتن را می‌بیند.

سرعت Register و RAM

☐ Registers :

بسیار سریع هستند؛ دسترسی به آن‌ها معمولاً در یک سیکل کلاک (یا کمتر) انجام می‌شود.

☐ Main Memory (RAM):

کندتر است؛ ممکن است چندین سیکل طول بکشد. همین باعث (stall) توقف موقت CPU می‌شود.

☐ Cache (حافظه نهان):

برای حل مشکل سرعت پایین RAM، یک حافظه سریع‌تر به نام Cache بین CPU و RAM قرار می‌گیرد. داده‌هایی که احتمال استفاده دوباره دارند را نگه می‌دارد تا CPU مجبور نشود همیشه به RAM سر بزند.

محافظت حافظه (Memory Protection)

سیستم‌عامل باید مکانیزم‌هایی داشته باشد که یک پردازنده نتواند به حافظه پردازنده دیگر دسترسی پیدا کند. این کار برای امنیت، پایداری و جلوگیری از خطا ضروری است.

مثال ساده :

1. فرض کن یک برنامه ساده داری که می‌خواهد مقدار $x = x + 1$ را اجرا کند: CPU مقدار x را از RAM می‌خواند.
3. اگر مقدار x قبلاً در Cache باشد → خیلی سریع گرفته می‌شود.
- اگر نباشد، CPU باید منتظر بماند تا از RAM آورده شود → stall رخ می‌دهد.
4. بعد از محاسبه، مقدار جدید دوباره در Register و سپس در Cache و RAM ذخیره می‌شود.

نکات کلیدی:

برنامه‌ها برای اجرا باید از دیسک → CPU Registers → RAM منتقل شوند. CPU فقط به Registers و RAM دسترسی مستقیم دارد. RAM کندتر از Register است و ممکن است CPU منتظر بماند (stall). Cache بین CPU و RAM قرار دارد تا سرعت را افزایش دهد. Memory Protection مانع دسترسی غیرمجاز پردازنده‌ها به حافظه یکدیگر می‌شود.

(Memory Management Requirements)

سیستم‌عامل باید هنگام مدیریت حافظه، چند نیاز کلیدی را برآورده کند تا پردازنده‌ها بتوانند به درستی و ایمن اجرا شوند. این نیازها عبارت‌اند از:

1. جابجایی (Relocation)

وقتی یک پردازنده از دیسک به حافظه آورده می‌شود، لزوماً همیشه در یک آدرس ثابت قرار نمی‌گیرد. سیستم‌عامل باید بتواند پردازنده‌ها را جابه‌جا کند تا فضای حافظه به شکل بهینه استفاده شود. مثال: اگر یک برنامه فکر کند همیشه از آدرس 1000 شروع می‌شود، در عمل سیستم‌عامل باید آن را به هر جای آزاد در حافظه منتقل کند و آدرس‌ها را تطبیق دهد.

2. محافظت (Protection)

پردازنده‌ها نباید به حافظه پردازنده‌های دیگر دسترسی داشته باشند. سیستم‌عامل مکانیزم‌هایی (مثل base و limit register یا MMU) فراهم می‌کند تا یک پردازنده فقط به بخش خودش دسترسی داشته باشد. بدون این محافظت، یک برنامه می‌تواند به طور تصادفی یا عمدی داده‌های برنامه دیگر یا خود سیستم‌عامل را خراب کند.

3. اشتراک‌گذاری (Sharing)

گاهی لازم است چند پردازنده یک بخش از حافظه را با هم به اشتراک بگذارند. مثال: چند برنامه می‌توانند یک کتابخانه (library) مشترک را استفاده کنند بدون اینکه هرکدام یک کپی جداگانه در حافظه داشته باشند. اشتراک‌گذاری باید کنترل‌شده و امن باشد.

4. سازمان‌دهی منطقی (Logical Organization)

حافظه به صورت خطی از آدرس صفر تا آخر RAM است. اما برنامه‌ها به صورت منطقی سازمان‌دهی می‌شوند: کد (Code) داده‌ها (Data) پشته (Stack) سیستم‌عامل باید این سازمان‌دهی منطقی را پشتیبانی کند.

5. سازمان‌دهی فیزیکی (Physical Organization)

سیستم‌عامل باید حافظه فیزیکی (RAM + حافظه ثانویه مانند دیسک) را مدیریت کند. داده‌ها بین حافظه اصلی (RAM) و حافظه جانبی (Disk) جابه‌جا می‌شوند (به خصوص در سیستم‌های با Virtual Memory). هدف: استفاده‌ی بهینه از RAM محدود و شبیه‌سازی حافظه‌ای بزرگ‌تر از آنچه واقعاً وجود دارد.

مثال ساده :

فرض کن دو برنامه همزمان در حال اجرا هستند:
برنامه A: یک ویرایشگر متن برنامه
برنامه B: یک مرورگر وب

سیستم‌عامل باید:

هرکدام را در بخش جداگانه حافظه قرار دهد (Relocation).
نگذارد مرورگر به داده‌های ویرایشگر دسترسی پیدا کند (Protection).

اگر هر دو از کتابخانه‌ی گرافیکی مشترکی استفاده می‌کنند، آن را به صورت اشتراکی در حافظه قرار دهد (Sharing). حافظه را طوری نمایش دهد که هر برنامه فکر کند آدرس‌هایش از صفر شروع می‌شوند (Logical Organization). داده‌های کمتر استفاده‌شده را روی دیسک قرار دهد و بخش فعال‌تر را در RAM نگه دارد (Physical Organization).

نکات کلیدی :

مدیریت حافظه باید ۵ نیاز اصلی را برآورده کند: Relocation, Protection, Sharing, Logical Organization, Physical Organization. این نیازها باعث می‌شوند حافظه ایمن، کارآمد و اشتراکی بین پردازنده‌ها استفاده شود.

1.(Relocation)

چرا به Relocation نیاز داریم؟
وقتی یک برنامه نوشته می‌شود، برنامه‌نویس نمی‌تواند از قبل بداند چه برنامه‌های دیگری همزمان در حافظه خواهند بود. در نتیجه، مشخص کردن یک آدرس ثابت برای برنامه غیرممکن یا بسیار محدودکننده است.

مشکل Swap in / Swap out

در سیستم‌های چندبرنامه‌ای (Multiprogramming)، ممکن است یک پردازنده از حافظه خارج (swap out) و روی دیسک ذخیره شود. وقتی دوباره به حافظه برمی‌گردد (swap in)، نمی‌توانیم او را مجبور کنیم دقیقاً به همان آدرس قبلی بازگردد. چون ممکن است آن بخش حافظه در این فاصله پر شده باشد. پس لازم است برنامه بتواند در هر جای آزاد از حافظه دوباره بارگذاری شود.

نقش سخت‌افزار و سیستم‌عامل

پردازنده در کدش از آدرس‌های منطقی (logical addresses) استفاده می‌کند (مثلاً "از آدرس 100 دستور را بگیر"). سیستم‌عامل و سخت‌افزار (به‌ویژه MMU = Memory Management Unit) این آدرس‌ها را به آدرس‌های فیزیکی واقعی (physical addresses) ترجمه می‌کنند. این ترجمه باعث می‌شود پردازنده متوجه جابه‌جایی‌اش نشود و همچنان درست کار کند.

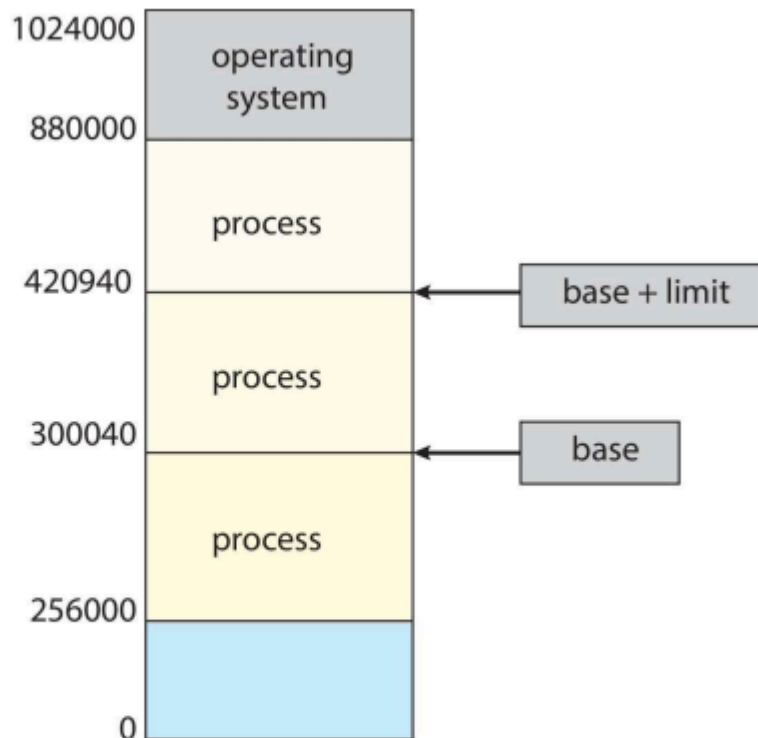
مثال ساده :

فرض کن برنامه‌ای داریم که فکر می‌کند در حافظه از آدرس 1000 شروع شده است. این برنامه دستور می‌دهد: "برو به آدرس 1005". اگر پردازنده واقعاً در حافظه فیزیکی از آدرس 4000 شروع شده باشد، MMU درخواست را به آدرس 4005 ترجمه می‌کند. بنابراین برنامه همچنان درست اجرا می‌شود، حتی اگر جایش در حافظه تغییر کند.

نکات کلیدی:

جابه‌جایی (Relocation) توانایی جابه‌جایی پردازنده‌ها در حافظه، بدون اینکه برنامه‌ها نیاز به تغییر داشته باشند. در سیستم‌های چندبرنامه‌ای و با Swap in/out بسیار ضروری است. Logical Address → Physical Address توسط سیستم‌عامل و سخت‌افزار (MMU) انجام می‌شود. پردازنده هیچ‌گاه آدرس واقعی‌اش در حافظه را نمی‌بیند؛ فقط با آدرس منطقی کار می‌کند.

2. (Protection)



نیاز به اطمینان از این است که یک فرآیند فقط می‌تواند به آدرس‌های موجود در فضای آدرس خود دسترسی داشته باشد. ما می‌توانیم این حفاظت را با استفاده از یک جفت ثابت پایه (base) و حد (limit) فراهم کنیم که فضای آدرس منطقی یک فرآیند را تعریف می‌کنند.

این اسلاید یکی از نیازمندی‌های اصلی مدیریت حافظه به نام حفاظت (Protection) را توضیح می‌دهد.

هدف حفاظت: جلوگیری از دسترسی یک فرآیند به حافظه اختصاص داده شده به فرآیندهای دیگر یا به سیستم عامل. این امر برای پایداری، امنیت و قابلیت اطمینان سیستم ضروری است.

روش پیاده‌سازی: روش ساده و کارآمد ارائه شده در این اسلاید استفاده از دو ثابت سخت‌افزاری خاص است:

1. ثابت پایه (Base Register): حاوی اولین آدرس فیزیکی (آدرس شروع) از محدوده حافظه‌ای است که به یک فرآیند خاص تخصیص داده شده است. هر زمان که فرآیند می‌خواهد به حافظه دسترسی پیدا کند، آدرس منطقی تولید شده توسط CPU به این مقدار پایه اضافه می‌شود تا آدرس فیزیکی واقعی محاسبه شود.
2. ثابت حد (Limit Register): حاوی اندازه محدوده حافظه تخصیص داده شده به فرآیند (مثلاً ۲۵۰۰۰۰ بایت) است. این ثابت تعیین می‌کند که یک فرآیند مجاز است تا چه حدی از حافظه اختصاص یافته خود را مورد دسترسی قرار دهد.

نحوه کار:

قبل از اجرای هر فرآیند، سیستم عامل مقادیر مناسب را در ثابت‌های پایه و حد بارگذاری می‌کند. برای هر دسترسی به حافظه توسط فرآیند، سخت‌افزار (MMU) به طور خودکار بررسی می‌کند که آدرس منطقی درخواستی از مقدار ثابت حد کمتر باشد. اگر آدرس در محدوده مجاز بود (یعنی $limit >$), آنگاه به آدرس پایه اضافه شده و به آدرس فیزیکی تبدیل می‌شود. اگر آدرس در محدوده مجاز نبود (یعنی $limit \leq$), یک خطا (حفاظت از حافظه یا segmentation fault) ایجاد می‌شود و دسترسی رد می‌شود.

مثال از نمودار:

جدول احتمالاً نشان می‌دهد که فضای آدرس یک فرآیند از آدرس فیزیکی ۲۵۰۰۰۰ شروع می‌شود ($base = 250000$) و اندازه آن ۲۵۰۰۰۰ واحد است ($limit = 250000$). بنابراین، این فرآیند فقط می‌تواند به آدرس‌های فیزیکی بین ۲۵۰۰۰۰ تا ۵۰۰۰۰۰ دسترسی داشته باشد. هر تلاشی برای دسترسی به آدرس‌های پایین‌تر (مثلاً ۲۴۹۹۹۹) یا بالاتر (مثلاً ۵۰۰۰۰۱) توسط سخت‌افزار مسدود خواهد شد.

3.(Sharing)

چرا به Sharing نیاز داریم؟

در بسیاری از مواقع چند پردازش (Process) نیاز دارند به یک بخش مشترک از حافظه دسترسی داشته باشند. مثال: چند برنامه ممکن است از یک کتابخانه‌ی استاندارد یا یک سرویس سیستم‌عامل استفاده کنند.

جلوگیری از هدررفت حافظه بدون اشتراک‌گذاری، هر پروژه مجبور بود یک کپی جداگانه از همان برنامه یا داده در حافظه داشته باشد. این کار باعث هدررفت حافظه می‌شود.

با Sharing، همه پروژه‌ها می‌توانند به همان نسخه مشترک دسترسی داشته باشند.

کنترل و امنیت در اشتراک‌گذاری اشتراک‌گذاری باید مدیریت‌شده باشد:

پروژه‌ها می‌توانند برنامه مشترک را فقط بخوانند (Read-Only).

پروژه‌ها نباید بتوانند محتوای مشترک را تغییر دهند (مگر اینکه سیستم‌عامل مجوز خاصی بدهد).

مثال ساده:

فرض کن همزمان سه برنامه‌ی مختلف (مثلاً Word, Excel, PowerPoint) در حال اجرا هستند.

هر سه به یک کتابخانه گرافیکی (Graphics Library) نیاز دارند.

به جای اینکه سه بار این کتابخانه در حافظه کپی شود، سیستم‌عامل آن را یک بار بارگذاری می‌کند. هر سه برنامه از همان نسخه استفاده می‌کنند (اشتراک‌گذاری حافظه).

نکات کلیدی:

اشتراک‌گذاری (Sharing) یعنی چند پروژه می‌توانند به یک بخش مشترک از حافظه دسترسی داشته باشند.

این کار باعث صرفه‌جویی در حافظه و افزایش کارایی می‌شود.

اشتراک‌گذاری معمولاً به صورت Read-Only انجام می‌شود تا پروژه‌ها نتوانند محتوای مشترک را تغییر دهند. کتابخانه‌ها (Libraries) و

سرویس‌های سیستم‌عامل از مهم‌ترین مثال‌های Sharing هستند.

4.(Logical Organization)

حافظه اصلی به صورت خطی حافظه اصلی (RAM) در سخت‌افزار به صورت یک فضای خطی از آدرس‌ها سازمان‌دهی می‌شود: 0, 1, 2, 3, ..., N. اما برنامه‌ها و کاربران معمولاً به شکل ماژولار (Module-Based) فکر می‌کنند، نه به شکل یک فضای خطی.

ماژول‌ها و استقلال آن‌ها

برنامه می‌تواند از چندین ماژول (Module) تشکیل شود (مثلاً کد اصلی، کتابخانه‌ها، داده‌ها). این ماژول‌ها می‌توانند به صورت مستقل نوشته و کامپایل شوند.

در زمان اجرا، سیستم‌عامل ارجاعات بین ماژول‌ها را حل می‌کند (Binding).

سطوح مختلف محافظت هر ماژول می‌تواند سطح دسترسی خاص خودش را داشته باشد:

Read Only → داده‌ها فقط قابل خواندن باشند (مثلاً جداول ثابت).

Execute Only → ماژول فقط اجرا شود (مثل کد برنامه).

Read/Write → برای داده‌های قابل تغییر.

اشتراک‌گذاری در سطح ماژول

ماژول‌ها می‌توانند بین پروژه‌ها به اشتراک گذاشته شوند.

چون کاربر هم معمولاً برنامه را در سطح ماژول می‌بیند، اشتراک‌گذاری در این سطح بسیار طبیعی و کاربرپسند است.

ابزار اصلی: Segmentation (تقسیم‌بندی)

بهترین ابزار برای پیاده‌سازی سازمان‌دهی منطقی Segmentation است.

در تقسیم‌بندی:

هر ماژول در یک Segment جدا قرار می‌گیرد.

سیستم‌عامل و سخت‌افزار (MMU) وظیفه دارند این Segments را مدیریت و آدرس‌دهی کنند.

مثال ساده :

فرض کن یک برنامه شامل:

کد اصلی (Main Code)

ماژول پردازش تصویر (Image Processing Module)

کتابخانه ریاضی (Math Library)

در روش Segmentation:

کد اصلی در یک Segment قرار می‌گیرد. ماژول پردازش تصویر در یک Segment دیگر.

کتابخانه ریاضی هم در یک Segment جدا که شاید با چند پروژه دیگر Shared باشد.

نکات کلیدی :

حافظه فیزیکی خطی است، اما کاربران برنامه را به صورت ماژولار می‌بینند.

سازمان‌دهی منطقی (Logical Organization) مکان می‌دهد هر ماژول مستقل نوشته و مدیریت شود. می‌توان سطوح مختلف دسترسی به هر

ماژول داد (Read, Write, Execute).

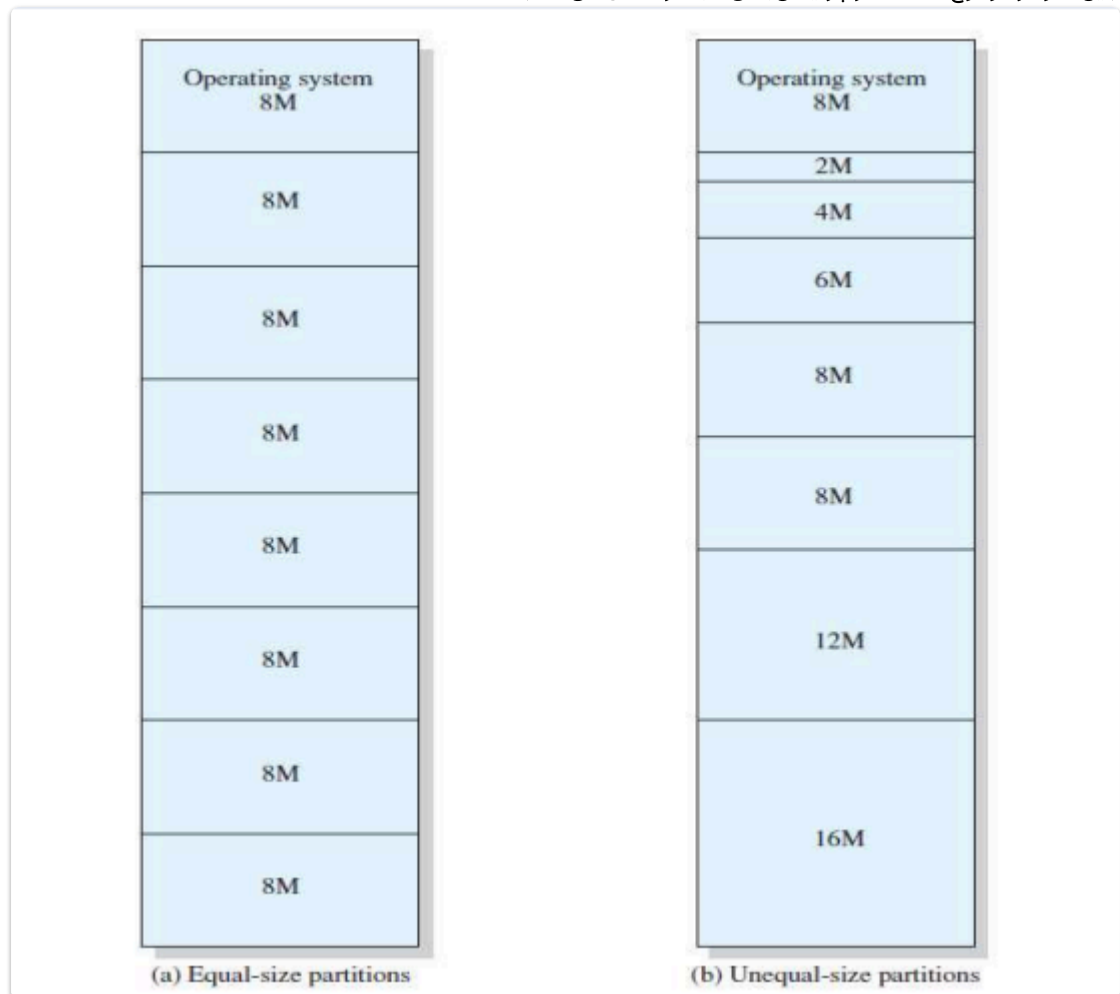
بهترین روش برای پیاده‌سازی Logical Organization ، Segmentation است.

memory management techniques

تکنیک	توضیحات	مزیت	عیب
تقسیم بندی ثابت	حافظه اصلی به چند پارتیشن پردازها ثابت تقسیم می‌شود فقط می‌توانند در یکی از این پارتیشن‌ها قرار بگیرند	ساده برای پیاده‌سازی سریار کم	Internal Fragmentation (اتلاف فضا) استفاده ناکارآمد از حافظه
تقسیم بندی پویا	پارتیشن‌ها به صورت پویا و متناسب با نیاز پردازها ساخته می‌شوند	بدون اتلاف داخلی (No internal fragmentation)	تکه‌تکه شدن خارجی (External Fragmentation)
صفحه بندی ساده	حافظه اصلی به فریم‌های مساوی تقسیم می‌شود. پردازها نیز به صفحات مساوی تقسیم می‌شوند.	بدون اتلاف خارجی (No external fragmentation)	اندکی اتلاف داخلی (Internal fragmentation)
قطعه بندی ساده	حافظه به قطعه‌هایی با اندازه متفاوت تقسیم می‌شود.	عدم اتلاف داخلی، منطقی‌تر برای برنامه‌نویس.	مشکل تکه‌تکه شدن خارجی
صفحه بندی حافظه مجازی	پردازها می‌توانند بزرگ‌تر از حافظه فیزیکی باشند بخش‌های موردنیاز در صورت لزوم به حافظه آورده می‌شوند	استفاده بهتر از حافظه، اجرای برنامه‌های بزرگ	پیچیدگی زیاد در مدیریت
قطعه بندی حافظه مجازی	ترکیبی از سگمنتیشن و حافظه مجازی	پشتیبانی از اشتراک‌گذاری و حفاظت قوی‌تر	سریار بیشتر در مدیریت

(Fixed Partitioning)

(این نمودار دو نوع مختلف از پارتیشن‌بندی ثابت را نشان می‌دهد)



شکل 7.2 مثال از پارتیشن‌بندی ثابت یک حافظه ۶۴ مگابایتی

(الف) پارتیشن‌های با اندازه برابر

(ب) پارتیشن‌های با اندازه نابرابر

این نمودار یکی از روش‌های اولیه مدیریت حافظه به نام پارتیشن‌بندی ثابت (Fixed Partitioning) را نشان می‌دهد. **مفهوم کلی:** در این روش، حافظه اصلی قبل از اجرای فرآیندی به چندین بخش مجزا و ثابت به نام پارتیشن تقسیم می‌شود. اندازه این پارتیشن‌ها در طول کار سیستم تغییر نمی‌کند.

انواع پارتیشن‌بندی ثابت:

1. پارتیشن‌های با اندازه برابر (الف):

در این روش، حافظه به پارتیشن‌هایی با اندازه یکسان تقسیم می‌شود.

در مثال ارائه شده، یک حافظه ۶۴ مگابایتی به هشت پارتیشن ۸ مگابایتی تقسیم شده است ($8M = 64M \div 8$).

مزیت: سادگی در پیاده‌سازی و مدیریت.

عیب: هدررفت داخلی (Internal Fragmentation) قابل توجه. اگر یک فرآیند فقط به ۲MB فضا نیاز داشته باشد، اما در یک پارتیشن ۸MB قرار گیرد، ۶MB از آن پارتیشن هدر می‌رود.

2. پارتیشن‌های با اندازه نابرابر (ب):

در این روش، حافظه به پارتیشن‌هایی با اندازه‌های مختلف تقسیم می‌شود.

در مثال ارائه شده، حافظه ۶۴ مگابایتی به پارتیشن‌های با اندازه‌های مختلف (مانند 2M, 4M, 6M, 8M, 12M, 16M) تقسیم شده است که جمع آنها ۶۴ مگابایت می‌شود.

مزیت: انعطاف‌پذیری بیشتر و کاهش هدررفت داخلی. می‌توان فرآیندهای کوچک را در پارتیشن‌های کوچک و فرآیندهای بزرگ را در پارتیشن‌های بزرگ قرار داد.

عیب: مدیریت آن پیچیده‌تر است. سیستم عامل باید تصمیم بگیرد که هر فرآیند را در کدام پارتیشن (که به اندازه کافی بزرگ باشد) قرار دهد.

کاربرد: این روش یکی از ساده‌ترین طرح‌های مدیریت حافظه بود و در سیستم‌عامل‌های اولیه مانند OS/360 IBM استفاده می‌شد.

پارتیشن‌بندی ثابت (Fixed Partitioning) چیست؟

در این روش، حافظه اصلی (RAM) در زمان راه‌اندازی سیستم (System Generation Time) به چند بخش ثابت (Partition) تقسیم می‌شود.

هر Partition فقط می‌تواند یک پردازش را نگه دارد.

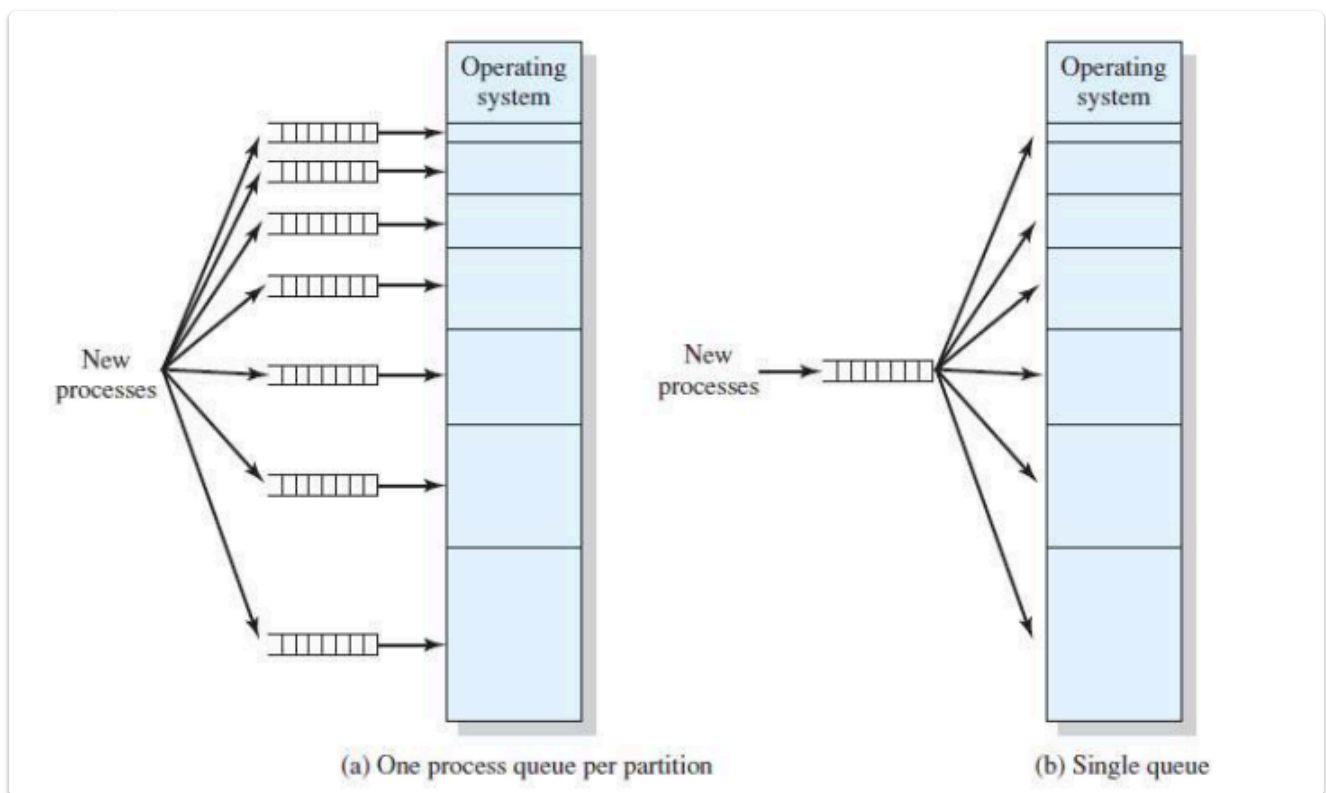
معایب روش پارتیشن‌بندی ثابت :

1. محدودیت تعداد پردازش‌ها چون تعداد Partition‌ها از قبل تعیین شده، تعداد پردازش‌های فعال در سیستم هم به همان تعداد محدود است. مثال: اگر حافظه به 4 پارتیشن تقسیم شده باشد، بیش از 4 پردازش همزمان نمی‌توانند فعال باشند.
2. استفاده ناکارآمد از حافظه (به‌ویژه برای پردازش‌های کوچک) اندازه‌ی Partition‌ها از قبل مشخص می‌شود. اگر یک برنامه کوچک‌تر از اندازه Partition باشد، فضای باقیمانده هدر می‌رود. مثال: اگر یک Partition به اندازه 100MB باشد و یک برنامه 40MB در آن قرار گیرد → 60MB استفاده‌نشده باقی می‌ماند.
3. پراکندگی داخلی (Internal Fragmentation) مشکل اصلی این روش است.
4. فضای بلااستفاده داخل یک Partition (به دلیل کوچک بودن پردازش نسبت به Partition) را Internal Fragmentation می‌نامند. این فضا برای پردازش دیگر قابل استفاده نیست، چون کل Partition به یک پردازش اختصاص یافته است.

نکات کلیدی:

پارتیشن‌بندی ثابت (Fixed Partitioning) باعث محدودیت در تعداد پردازش‌های فعال می‌شود. اندازه‌ی ثابت Partition‌ها استفاده‌ی بهینه از RAM را دشوار می‌کند. مشکل اصلی این روش است. این معایب باعث شد سیستم‌های مدرن به سمت Dynamic Partitioning، Paging و Segmentation حرکت کنند.

placement algorithm



شکل 7.3 تخصیص حافظه برای پارتیشن‌بندی ثابت

- (الف) یک صف فرآیند به ازای هر پارتیشن
- (ب) یک صف واحد

این نمودار دو سیاست (استراتژی) مختلف برای زمان‌بندی و تخصیص حافظه در یک سیستم با پارتیشن‌بندی ثابت (Fixed Partitioning) را نشان می‌دهد.

در پارتیشن‌بندی ثابت، حافظه اصلی به چندین پارتیشن (بخش) با اندازه‌های ثابت و از پیش تعریف شده تقسیم می‌شود.

- **روش (الف):** یک صف فرآیند به ازای هر پارتیشن - در این روش، برای هر پارتیشن یک صف جداگانه از فرآیندهای آماده برای اجرا وجود دارد. فرآیندهای جدید بر اساس اندازه‌شان به صف مربوط به پارتیشن با اندازه مناسب وارد می‌شوند. به عنوان مثال، یک فرآیند کوچک فقط در صف مربوط به پارتیشن‌های کوچک قرار می‌گیرد.
- **مزیت:** سادگی در مدیریت.

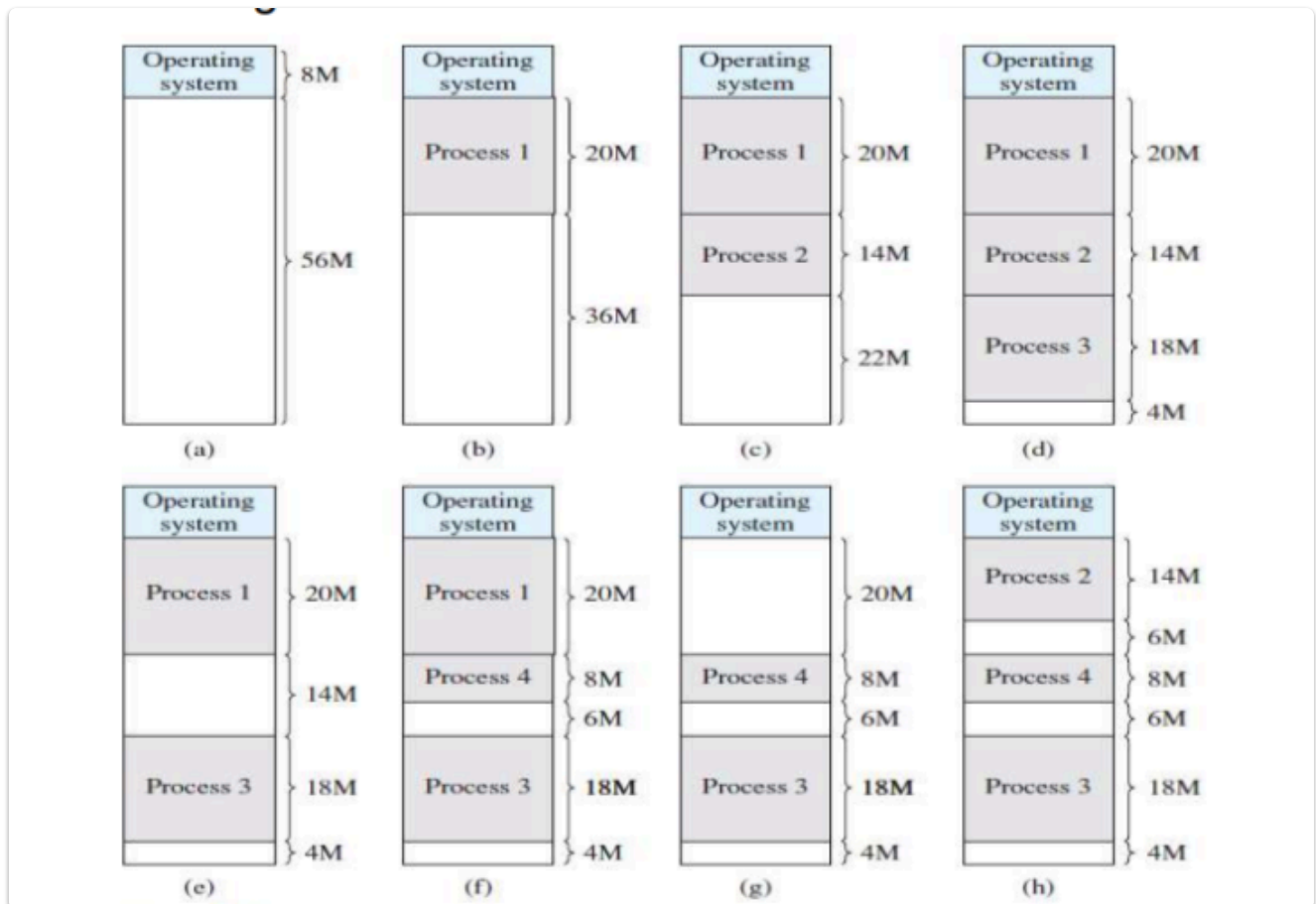
عیب: می‌تواند منجر به هدررفت حافظه (internal fragmentation) شود اگر اندازه فرآیند بسیار کوچک‌تر از اندازه پارتیشن باشد. همچنین ممکن است یک پارتیشن خالی باشد در حالی که فرآیندهای موجود در صف یک پارتیشن دیگر در حال انتظار هستند.

- **روش (ب):** یک صف واحد در این روش، تنها یک صف واحد برای تمامی فرآیندهای آماده برای اجرا وجود دارد. هنگامی که یک پارتیشن آزاد می‌شود، سیستم عامل فرآیند بعدی در صف واحد را که اندازه آن کوچک‌تر یا مساوی پارتیشن آزاد شده است، برای اجرا در آن پارتیشن انتخاب می‌کند. این روش انعطاف‌پذیری بیشتری دارد و معمولاً با استفاده از یک الگوریتم جایگذاری (Placement Algorithm) مانند "اولین مناسب (First-Fit)" یا "بهترین مناسب (Best-Fit)" پیاده‌سازی می‌شود تا تصمیم بگیرد کدام فرآیند و کدام پارتیشن را تخصیص دهد.
- **مزیت:** استفاده کارآمدتر از حافظه و کاهش احتمال idle ماندن پارتیشن‌ها.
- **عیب:** پیچیدگی بیشتر در مدیریت و زمان‌بندی.

خلاصه: این نمودار تضاد بین سادگی (روش الف) و کارایی (روش ب) در مدیریت حافظه را نشان می‌دهد.

Dynamic partitioning

پارتیشن‌ها دارای طول و تعداد متغیر هستند. به هر پردازش دقیقاً به اندازه‌ای حافظه اختصاص داده می‌شود که نیاز دارد. تکه‌تکه شدن خارجی (External Fragmentation) وجود دارد.



- **شکل 7.4** اثر پارتیشن بندی پویا (Dynamic Partitioning)

این شکل چند مرحله تخصیص حافظه را نشان می‌دهد:

a(:

سیستم عامل 8 مگابایت را اشغال کرده و 56 مگابایت آزاد باقی مانده است.

b(:

پردازش 1 (Process 1) به اندازه 20 مگابایت وارد حافظه شده، 36 مگابایت آزاد مانده است.

c(:

پردازش 2 با اندازه 14 مگابایت وارد می‌شود. حالا 22 مگابایت فضای آزاد داریم.

d(:

پردازش 3 با اندازه 18 مگابایت وارد می‌شود. 4 مگابایت فضای آزاد باقی مانده است.

e(:

پردازش 2 پایان می‌یابد، فضای 14 مگابایتی آزاد می‌شود (فضای آزاد بین پردازش‌ها قرار دارد → تکه‌تکه شدن خارجی).

f(:

پردازش 4 با اندازه 6 مگابایت در آن فضای 14 مگابایتی قرار می‌گیرد، 8 مگابایت آزاد باقی می‌ماند.

g(:

پردازه 5 با اندازه 6 مگابایت وارد همان فضای 8 مگابایتی می‌شود، 2 مگابایت آزاد باقی می‌ماند.

h(:

در پایان چند بخش کوچک آزاد داریم: 4 مگابایت در انتها + 2 مگابایت در وسط → که نشان‌دهنده‌ی تکه‌تکه شدن خارجی است.

Placement algorithm

در Dynamic Partitioning، حافظه به صورت پویا به پردازه‌ها اختصاص داده می‌شود. اما وقتی یک پردازه جدید وارد می‌شود، سیستم‌عامل باید تصمیم بگیرد کدام بلوک آزاد حافظه (Free Block) را به آن بدهد.

برای این تصمیم، چند الگوریتم اصلی وجود دارد:

1. Best-Fit

ایده: بلوکی از حافظه انتخاب می‌شود که از نظر اندازه نزدیک‌ترین به اندازه پردازه باشد. مزیت: کمترین مقدار فضای هدررفته (باقیمانده). عیب: نیاز به جستجوی کل لیست بلوک‌های آزاد دارد → زمان بیشتری می‌برد. مشکل: باعث می‌شود تعداد زیادی بلوک‌های خیلی کوچک باقی بمانند که دیگر قابل استفاده نیستند.

2. First-Fit

ایده: جستجو از ابتدای حافظه شروع می‌شود و اولین بلوک آزاد به اندازه کافی بزرگ به پردازه داده می‌شود. مزیت: سریع‌تر از Best-Fit است (چون اولین بلوک مناسب انتخاب می‌شود). عیب: تمایل دارد حافظه آزاد کوچک در ابتدای حافظه باقی بگذارد → پراکندگی (External Fragmentation).

3. Next-Fit

ایده: مثل First-Fit است، با این تفاوت که جستجو از آخرین محل تخصیص انجام می‌شود (نه از ابتدای حافظه). مزیت: تخصیص یکنواخت‌تر در حافظه. عیب: در عمل کمی ضعیف‌تر از First-Fit عمل می‌کند (بیشتر fragmentation ایجاد می‌کند).

مثال ساده :

فرض کن حافظه آزاد شامل بلوک‌های: 100K, 500K, 200K, 300K, 600K است.

پردازه‌ای با اندازه 212K می‌خواهد وارد حافظه شود:

First-Fit → 500K بلوک انتخاب می‌شود.

Best-Fit → 300K بلوک انتخاب می‌شود (212 به همه نزدیک‌تر به 300).

Next-Fit → 200K بلوک انتخاب می‌شود، حالا بلوک 300K اگر آخرین تخصیص در بلوک 200K.

نکات کلیدی:

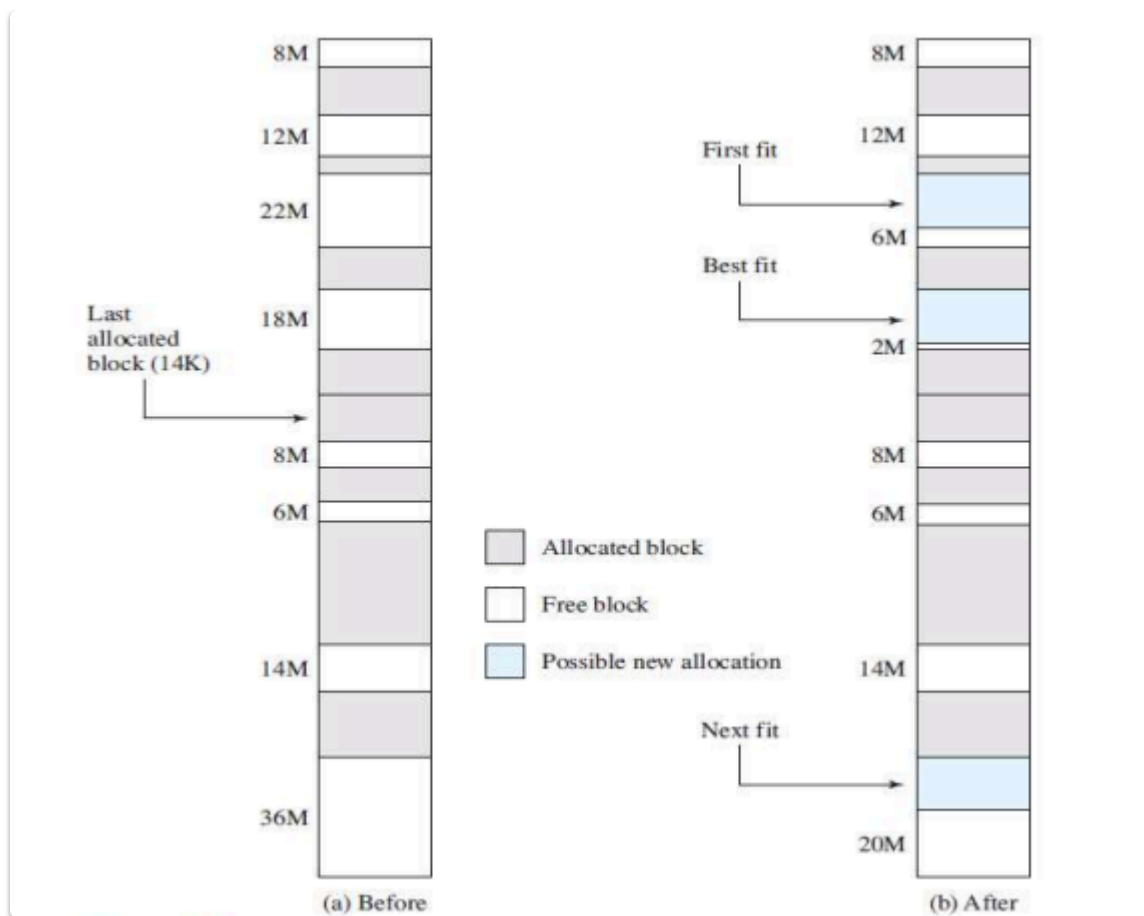
Best-Fit → استفاده بهینه‌تر از فضا، ولی سرعت پایین‌تر و ایجاد بلوک‌های کوچک غیرقابل استفاده.

First-Fit → سریع‌تر، اما معمولاً External Fragmentation بیشتری در ابتدا ایجاد می‌کند.

Next-Fit → تخصیص یکنواخت‌تر، ولی عملکرد کمی بدتر از First-Fit.

انتخاب الگوریتم بستگی به کارایی مورد انتظار و ویژگی بار کاری سیستم دارد.

(Placement Algorithm)



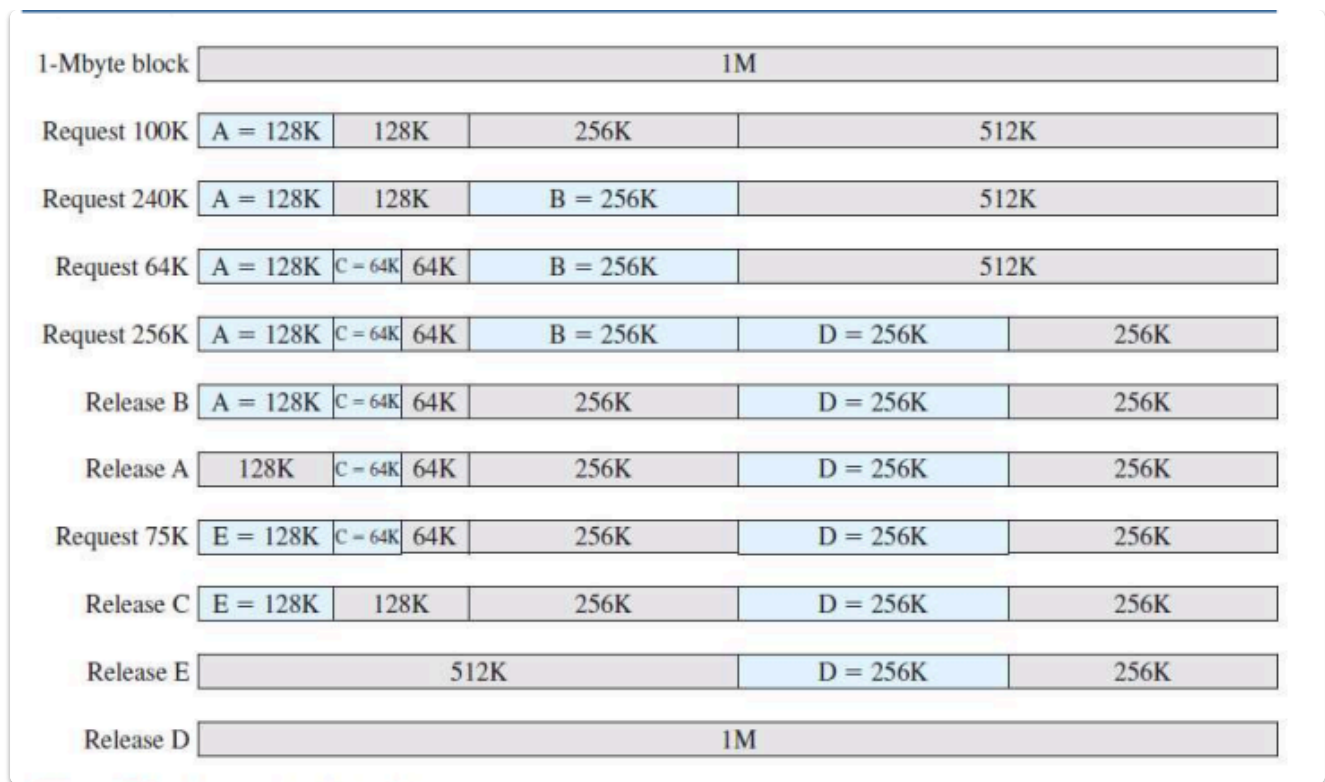
شکل 7.5 نمونه پیکربندی حافظه قبل و بعد از تخصیص یک بلوک ۱۶ مگابایتی

این نمودار سه الگوریتم معروف برای انتخاب یک بلوک از لیست حافظه آزاد هنگام درخواست تخصیص حافظه (در اینجا برای یک بلوک ۱۶ مگابایتی) را مقایسه می‌کند. وضعیت اولیه (قبل): چندین بلوک حافظه آزاد با اندازه‌های مختلف وجود دارد (8M, 12M, 22M, 18M, 8M, 6M, 14M, 36M). یک بلوک تخصیص یافته (14K) نیز وجود دارد.

الگوریتم‌های جایگذاری:

- اولین مناسب (First Fit):** این الگوریتم از ابتدای لیست حافظه آزاد شروع به جستجو می‌کند و اولین بلوک آزادی که به اندازه کافی بزرگ باشد ($16M \leq$) را برای تخصیص انتخاب می‌کند. در این مثال، اولین بلوک مناسب، بلوک 22M است. (این انتخاب با برچسب "First fit" و یک فلش به سمت بلوک 22M نشان داده شده است).
- بهترین مناسب (Best Fit):** این الگوریتم تمام لیست حافظه آزاد را جستجو می‌کند و کوچکترین بلوک آزادی که درخواست را برآورده می‌کند (یعنی کوچکترین بلوک $16M \leq$) را انتخاب می‌کند. هدف کاهش هدررفت حافظه (fragmentation داخلی) است. در این مثال، بهترین بلوک، بلوک 18M است، زیرا از بین بلوک‌های بزرگ‌تر یا مساوی 16M، کوچکترین است. (این انتخاب با برچسب "Best fit" و یک فلش به سمت بلوک 18M نشان داده شده است).
- مناسب بعدی (Next Fit):** این الگوریتم شبیه First Fit است، اما به جای شروع همیشه از ابتدای لیست، جستجو را از محل آخرین تخصیص (در اینجا بلوک 14K) ادامه می‌دهد. در این مثال، جستجو از آخرین تخصیص (14K) ادامه می‌یابد و اولین بلوک مناسب بعد از آن، بلوک 22M یا 18M (بسته به ترتیب لیست) خواهد بود. (این الگوریتم با برچسب "Next fit" در نمودار نشان داده شده است).

وضعیت نهایی (بعد): پس از تخصیص، بلوک انتخاب شده (مثلاً 22M) به دو بخش تقسیم می‌شود: یک بخش 16M که به فرآیند اختصاص می‌یابد (بلوک تخصیص داده شده). یک بخش باقیمانده (مثلاً $22M - 16M = 6M$) که به عنوان یک بلوک آزاد جدید به لیست اضافه می‌شود. (این موضوع با اعداد 2M, 6M, 2M, 8M, 6M در نمودار نشان داده شده که نشان‌دهنده اندازه‌های بلوک‌های آزاد جدید پس از تخصیص هستند).



• شکل 7.6 مثال سیستم جفتی (Buddy System)

هر دو طرح پارتیشن‌بندی ایستا (Fixed) و پویا (Dynamic) معایبی دارند:

- **پارتیشن‌بندی ایستا:** تعداد پرده‌های فعال را محدود می‌کند و ممکن است در صورت عدم تطابق بین اندازه پارتیشن‌های موجود و اندازه پرده‌ها، منجر به استفاده ناکارآمد از فضا شود.
- **پارتیشن‌بندی پویا:** نگهداری و مدیریت آن پیچیده‌تر است و سربار فشرده‌سازی (Compaction) را نیز شامل می‌شود.
- در سیستم بادی، بلوک‌های حافظه به اندازه کلمه هستند، که در آن:
- کوچک‌ترین اندازه بلوکی که تخصیص داده می‌شود
- بزرگ‌ترین اندازه بلوکی که تخصیص داده می‌شود؛ معمولاً کل اندازه حافظه موجود برای تخصیص است.

درخواست‌ها و آزادسازی‌ها:

1 مگابایت بلوک حافظه در ابتدا در دسترس است.

درخواست 100K → بلوک 128K تخصیص داده می‌شود (A = 128K). باقی‌مانده: 128K, 256K, 512K درخواست 240K → بلوک 256K تخصیص داده می‌شود (B = 256K). باقی‌مانده: 128K, 128K, 512K درخواست 64K → بلوک 128K نصف می‌شود (E = 64K). باقی‌مانده: 64K, 128K, 256K, 512K

درخواست 256K → بلوک 256K تخصیص داده می‌شود (D = 256K). باقی‌مانده: 128K, 64K, 64K, 512K آزادسازی B → بلوک 256K آزاد می‌شود.

آزادسازی A → بلوک 128K آزاد می‌شود.

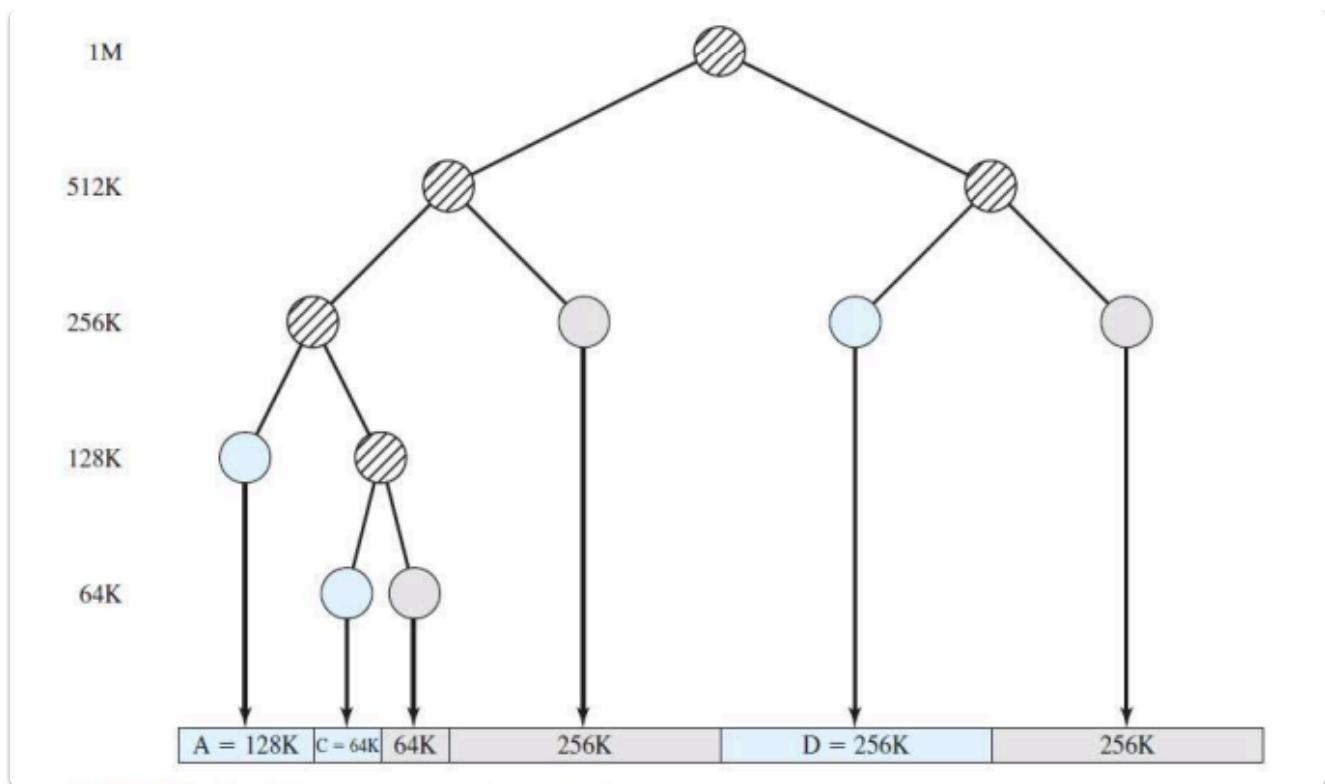
آزادسازی 75K → همان بلوک 128K قبلی آزاد می‌شود (چون نزدیک به 100K بوده).

آزادسازی E → بلوک 64K آزاد می‌شود.

آزادسازی D → بلوک 256K آزاد می‌شود.

در نهایت همه بلوک‌ها با هم ادغام می‌شوند و یک بلوک 1M آزاد خواهیم داشت.

Buddy System

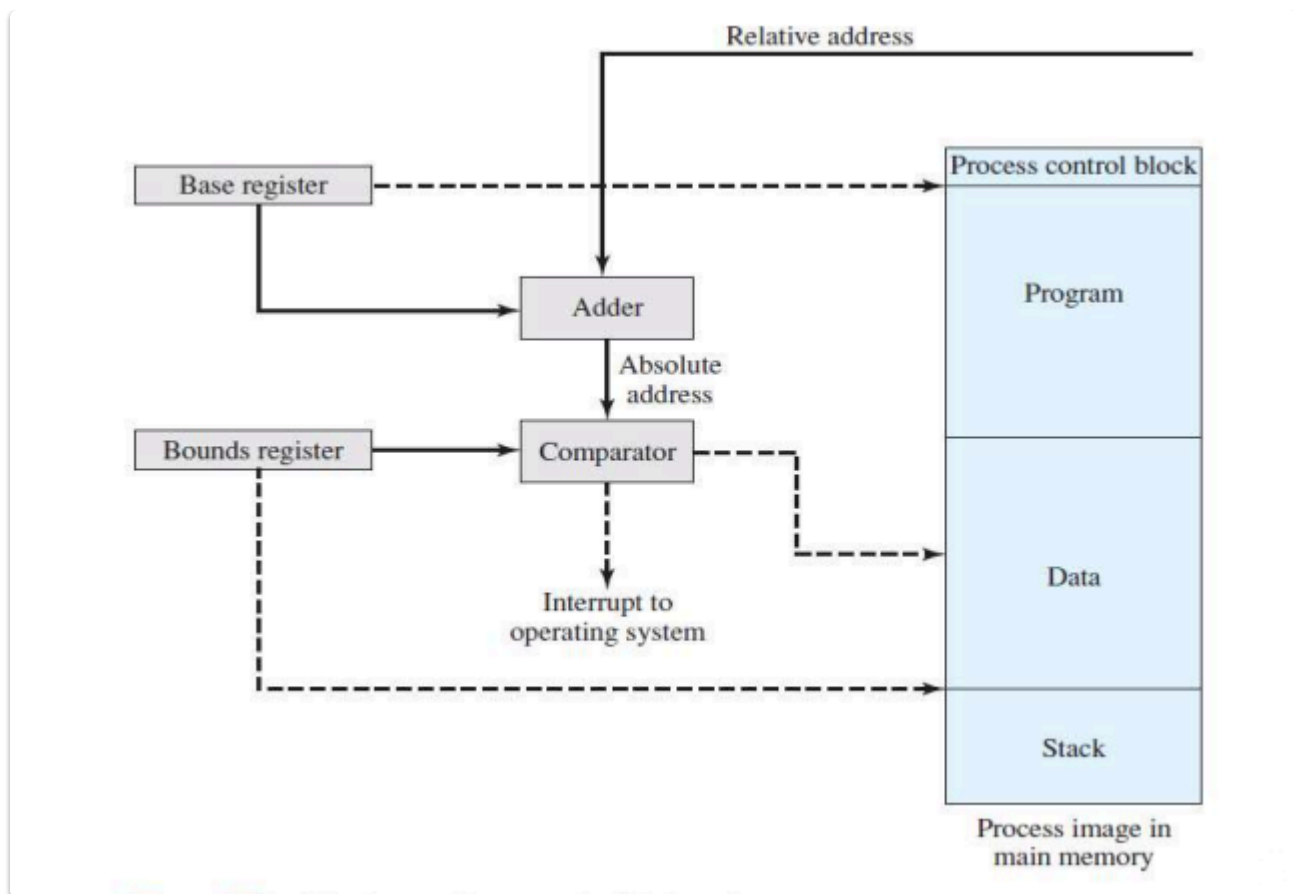


درخت دودویی بلوک‌های حافظه را نشان می‌دهد:
 در ریشه: 1M تقسیم به دو نیم: 512K + 512K سپس به 256K + 256K تقسیم می‌شوند.
 به همین ترتیب تا بلوک‌های کوچک‌تر (مانند 128K و 64K) تقسیم ادامه می‌یابد.
 گره‌ها نشان‌دهنده بلوک‌های تخصیص یافته یا آزاد هستند (مثل A = 128K، E = 64K، D = 256K و ...).

Addresses

آدرس‌ها (Addresses):

- آدرس منطقی (Logical Address): مرجعی به یک محل حافظه است که مستقل از تخصیص جاری داده‌ها در حافظه می‌باشد. پیش از دسترسی به حافظه، باید ترجمه به یک آدرس فیزیکی انجام گیرد.
- آدرس نسبی (Relative Address): نوع خاصی از آدرس منطقی است که در آن، آدرس به صورت نسبی نسبت به یک نقطه شناخته‌شده بیان می‌شود (مثلاً یک مقدار در ثبات پردازنده).
- آدرس فیزیکی (Physical Address): یا آدرس مطلق، همان مکان واقعی در حافظه اصلی است.



شکل 7.8 پشتیبانی سخت‌افزاری برای جابجایی (انتقال)

- ثبات پایه (Base register)
- جمع‌کننده (Adder)
- آدرس مطلق (Absolute address)
- مقایسه‌گر (Comparator)
- وقفه به سیستم عامل (Interrupt to operating system)
- بلوک کنترل فرآیند (Process control block)
- برنامه (Program)
- داده (Data)
- پشته (Stack)

این نمودار معماری سخت‌افزاری مورد نیاز برای پیاده‌سازی دو مفهوم مهم را نشان می‌دهد: جابجایی (Relocation) و حفاظت (Protection).

اجزا و نحوه کار:

1. ثبات پایه (Base Register): این ثبات، آدرس فیزیکی شروع (پایه) ناحیه‌ای از حافظه را که به یک فرآیند خاص تخصیص داده شده است، نگهداری می‌کند.
2. ثبات حد (Limit Register - که در نمودار اصلی implied است): این ثبات (که احتمالاً در کنار مقایسه‌گر قرار دارد)، حداکثر اندازه مجاز برای آدرس‌های منطقی آن فرآیند را نگهداری می‌کند. این ثبات در بلوک کنترل فرآیند (PCB) ذخیره می‌شود و هنگام اجرای فرآیند در ثبات سخت‌افزاری مربوطه بارگذاری می‌گردد.
3. آدرس منطقی (Logical Address): این آدرس توسط واحد پردازش مرکزی (CPU) و در حین اجرای دستورالعمل‌های یک فرآیند تولید می‌شود. این آدرس relative (نسبی) است و از پایه حافظه فرآیند شروع می‌شود.
4. مقایسه‌گر (Comparator): این جزء سخت‌افزاری آدرس منطقی تولید شده را با مقدار ثبات حد مقایسه می‌کند. اگر آدرس منطقی از حد مجاز بزرگ‌تر یا مساوی باشد، به معنای نقض دسترسی است.
5. وقفه به سیستم عامل (Interrupt): اگر مقایسه‌گر یک دسترسی غیرمجاز را تشخیص دهد (آدرس > limit)، یک وقفه (interrupt) برای سیستم عامل ارسال می‌کند. سیستم عامل سپس فرآیند متخلف را معمولاً terminates می‌کند (این خطا اغلب به عنوان segmentation fault شناخته می‌شود).
6. جمع‌کننده (Adder): اگر آدرس منطقی معتبر بود (آدرس < limit)، این جزء سخت‌افزاری، آدرس منطقی را با مقدار ثبات پایه جمع می‌کند تا آدرس مطلق (فیزیکی) واقعی در حافظه اصلی را محاسبه کند. آدرس فیزیکی = آدرس منطقی + مقدار ثبات پایه

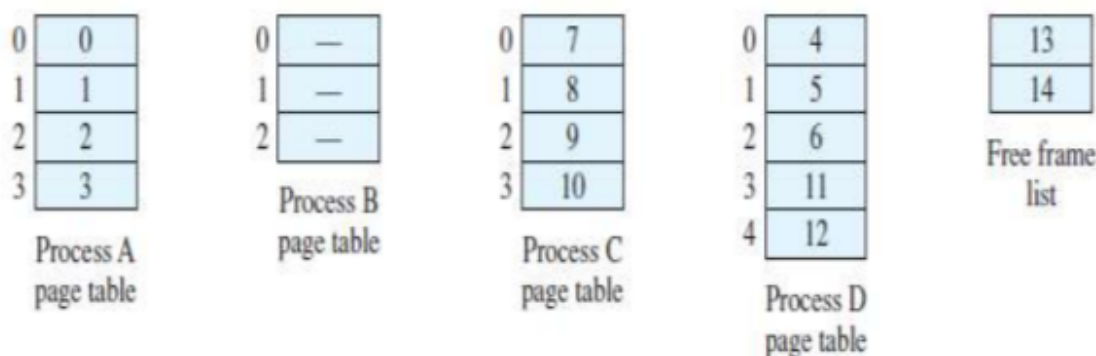
7. بلوک کنترل فرآیند (PCB): این ساختار داده سیستمی، اطلاعات مهم مربوط به هر فرآیند از جمله مقادیر صحیح base و limit برای آن فرآیند را ذخیره می‌کند. هنگامی که یک فرآیند برای اجرا انتخاب می‌شود، سیستم عامل این مقادیر را از PCB آن into the base and limit registers بارگذاری می‌کند.

خلاصه عملکرد:

این مکانیسم سخت‌افزاری به هر فرآیند این illusion (توهم) را می‌دهد که کل حافظه را در اختیار دارد (با شروع از آدرس صفر)، در حالی که سیستم عامل و سخت‌افزار به طور خودکار آدرس‌های آن را به مکان فیزیکی واقعی خود جابجا (Relocate) می‌کنند و همزمان از دسترسی به حافظه دیگر فرآیندها محافظت (Protection) به عمل می‌آورند.

paging

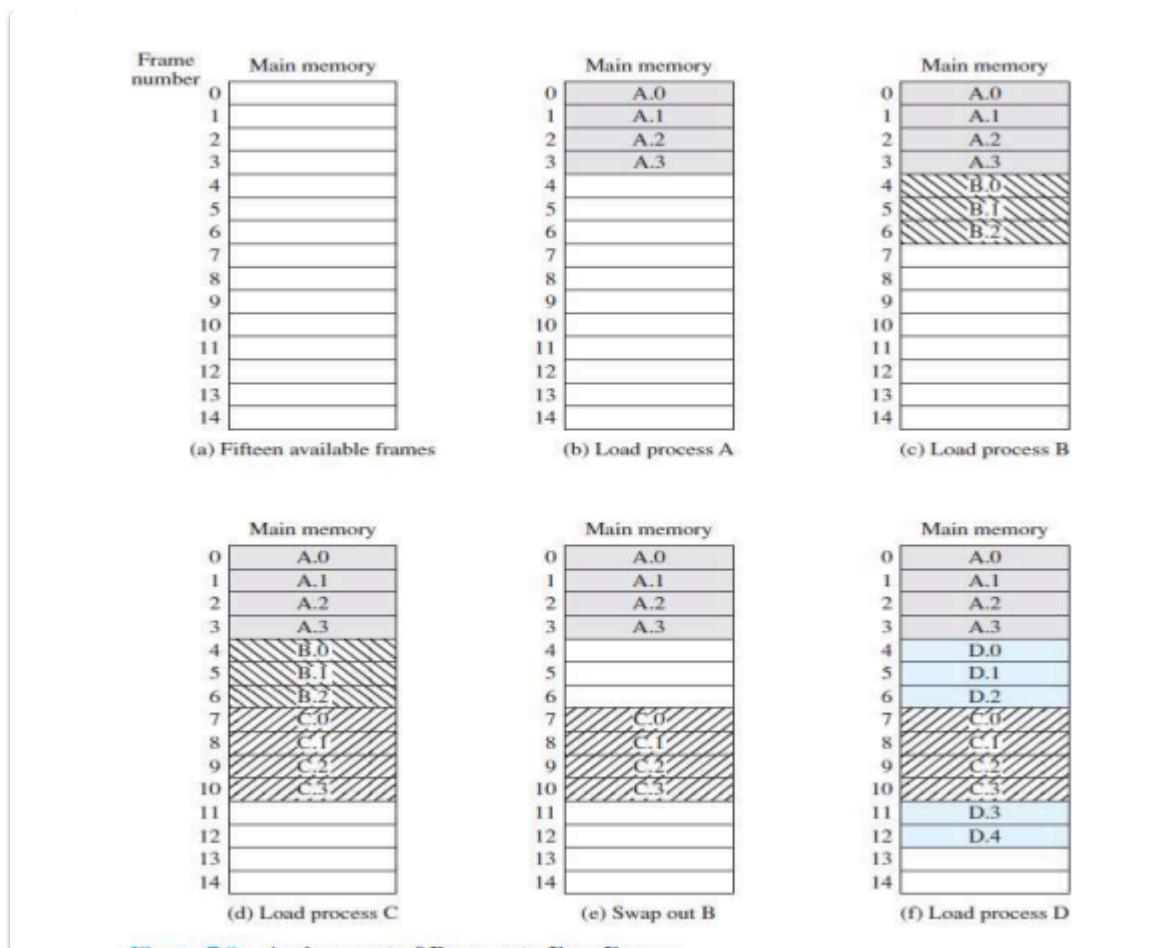
- یک جدول صفحه (page table) شامل یک ورودی برای هر صفحه از فرآیند است، به طوری که این جدول به راحتی توسط شماره صفحه (که از صفحه ۰ شروع می‌شود) فهرست‌بندی (ایندکس) می‌شود.
- هر ورودی جدول صفحه (page table entry) حاوی شماره قاب (فریم) در حافظه اصلی است (در صورت وجود) که صفحه مربوطه را در خود نگه می‌دارد.



شکل 7.10 ساختارهای داده برای مثال شکل 7.9 در مقطع زمانی (f) (Epoch)

این نمودار داده‌ساختارهای مورد استفاده در مدیریت حافظه مجازی با روش صفحه‌بندی را نشان می‌دهد.

1. سمت چپ (Main Memory): نشان‌دهنده حافظه اصلی فیزیکی است که به قاب‌های صفحه (page frames) تقسیم شده است. هر ستون احتمالاً نشان‌دهنده این است که محتوای هر قاب حافظه به کدام فرآیند و کدام صفحه مجازی تعلق دارد (مثلاً قاب ۰ از حافظه، صفحه ۰ از Process A را در خود نگه داشته است).
2. لیست قاب‌های آزاد (Free frame list): فهرستی از قاب‌های حافظه که در حال حاضر استفاده نمی‌شوند و آزاد هستند.
3. سمت راست: جدول‌های صفحه (page tables) برای چهار فرآیند مختلف (A, B, C, D) را نشان می‌دهد. هر جدول صفحه یک آرایه است که برای هر صفحه مجازی یک فرآیند، شماره قاب فیزیکی که آن صفحه در آن قرار دارد را نگاشت می‌کند (ذخیره می‌کند).
4. مقطع زمانی (f) (Epoch): به این معنی است که این نمودار وضعیت سیستم را در یک لحظه خاص از زمان (که در مثال اصلی با حرف f مشخص شده) نشان می‌دهد.



شکل 7.9 اختصاص فرآیندها به قاب‌های آزاد

این نمودار توالی وقایع در مدیریت حافظه مجازی را در شش مرحله (از الف تا و) نشان می‌دهد:

1. ستون (الف): وضعیت اولیه حافظه را نشان می‌دهد. ۱۵ قاب (Frame) حافظه اصلی همگی خالی و آزاد هستند.
2. ستون (ب): فرآیند A در حافظه بارگذاری (Load) شده است. صفحات این فرآیند (A.0 تا A.14) در قاب‌های حافظه قرار گرفته‌اند.
3. ستون (ج): فرآیند B نیز بارگذاری شده است. از آنجایی که حافظه پر است، برخی از صفحات فرآیند A (مثلاً صفحه A.1) از حافظه خارج شده و صفحات فرآیند B (مثلاً B.0 و B.1) جایگزین آنها شده‌اند. این نمودار حالت خاصی از جایگزینی صفحه را نشان می‌دهد.
4. ستون (د): فرآیند C بارگذاری می‌شود و جایگزین برخی صفحات دیگر (احتمالاً از فرآیندهای A یا B) می‌شود.
5. ستون (ه): فرآیند B به طور کامل از حافظه اصلی خارج (Swap out) می‌شود. قاب‌هایی که متعلق به B بودند now آزاد شده‌اند.
6. ستون (و): فرآیند D در قاب‌های آزاد شده بارگذاری می‌شود.

نکات مهم:

- A.0: است «A» به معنای «صفحه ۰ از فرآیند A».
- swap در اینجا به احتمال زیاد به معنای خارج کردن صفحات یک فرآیند از حافظه اصلی و انتقال آن به فضای (Swapping) تعویض روی دیسک است.
- هدف این نمودار نشان دادن پویایی مدیریت حافظه و چگونگی پر و خالی شدن قاب‌ها توسط سیستم عامل است.

(Segmentation)

تقسیم‌بندی منطقی حافظه (Segmentation) یک تکنیک مدیریت حافظه است که سازمان‌دهی منطقی برنامه‌ها و داده‌ها را تسهیل می‌کند.

مفهوم Segmentation

برنامه کاربر و داده‌های مرتبط با آن به چند Segment (بخش) تقسیم می‌شوند. هر Segment می‌تواند یک بخش خاص برنامه یا داده باشد:

- ☐ Code Segment → کد برنامه
- ☐ Data Segment → داده‌های برنامه
- ☐ Stack Segment → پشته

□ Library Segment → کتابخانه‌های مورد استفاده

طول Segments

یکسان نیست و می‌تواند متفاوت باشد Segments طول، Paging برخلاف

به اندازه واقعی نیاز برنامه باشد، بدون Segment تعیین می‌شود. این انعطاف باعث می‌شود که هر Segment فقط یک حداکثر طول هدررفت زیاد.

شباهت با Dynamic Partitioning

چون Segments اندازه‌های متفاوت دارند، این روش شبیه به Dynamic Partitioning است: بلوک‌ها هم اندازه نیستند.

هر Segment می‌تواند در هر قسمت خالی حافظه جای بگیرد.

به همین دلیل، Segmentation می‌تواند با Paging ترکیب شود (Segmentation + Paging) تا مزایای هر دو روش را داشته باشد.

مثال ساده

فرض کن یک برنامه با این Segments داریم:

- Code = 50 KB
 - Data = 120 KB
 - Stack = 20 KB
 - Library = 80 KB
- حافظه فیزیکی می‌تواند به صورت پراکنده باشد:
- Code → Memory Block 2
 - Data → Memory Block 4
 - Stack → Memory Block 1
 - Library → Memory Block 3
- مزیت: هدررفت حافظه کمتر و سازمان‌دهی منطقی برنامه بهتر است.

نکات کلیدی

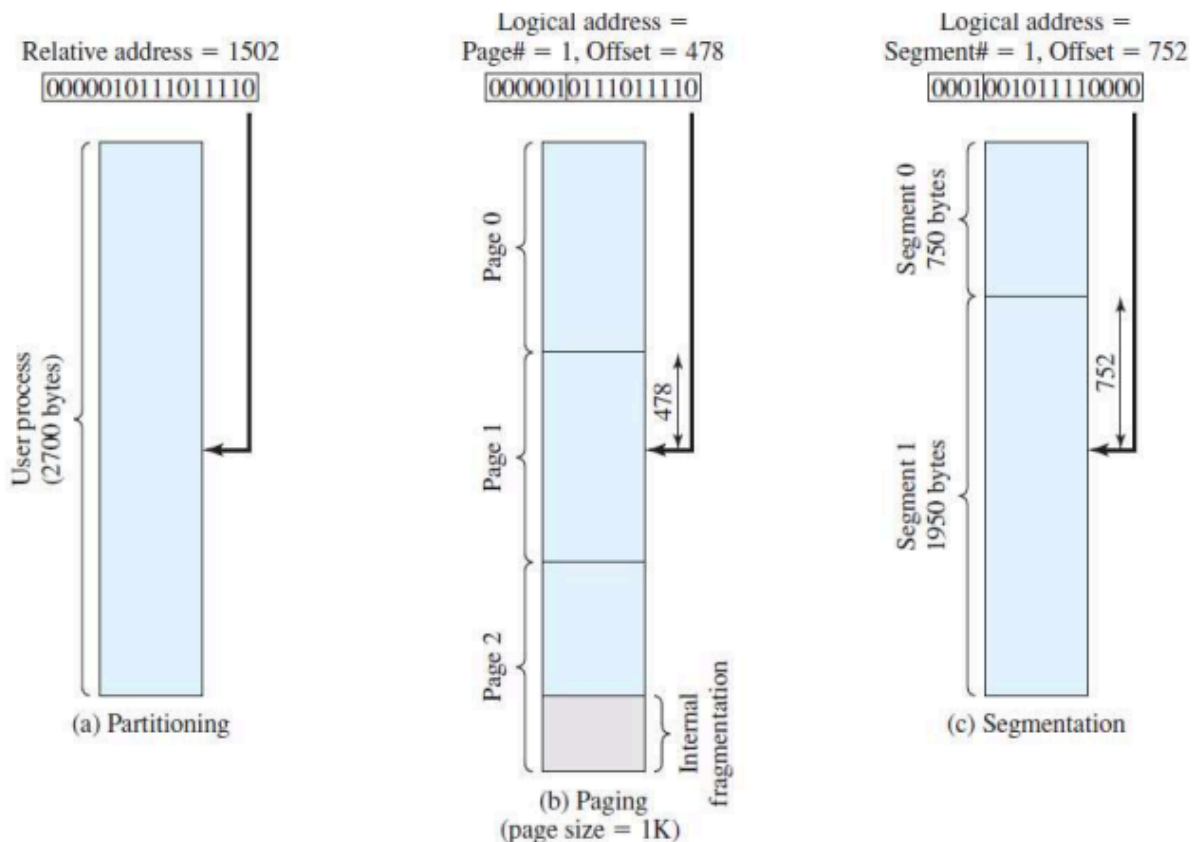
تقسیم‌بندی منطقی حافظه (Segmentation) برنامه را به چند Segment با اندازه متغیر تقسیم می‌کند.

هر Segment می‌تواند در هر قسمت آزاد حافظه قرار گیرد.

شباهت زیادی به Dynamic Partitioning دارد.

Segmentation + Paging → مدیریت حافظه بالا در کارایی و کارایی بالا در مدیریت حافظه.

(Logical Address)



شکل 7.11 آدرس‌های منطقی

(این بخش سه روش مختلف برای تفسیر یک آدرس حافظه مجازی (1502) را نشان می‌دهد)

- **آدرس نسبی** = 1502 [0000010111011110] (نمایش باینری آدرس 1502)
- **(الف) تقسیم‌بندی (Partitioning)** (در این روش آدرس به بخش‌های جدا تقسیم نمی‌شود و به عنوان یک عدد واحد در نظر گرفته می‌شود)
- **آدرس منطقی** = شماره صفحه = 1، افسست (تغییر) = 478 [0000010111011110] (نمایش باینری آدرس 1502) (بیت‌های این آدرس به دو بخش تقسیم شده‌اند: بخشی برای شماره صفحه و بخشی برای افسست)
- **(ب) صفحه‌بندی (Paging)** (اندازه صفحه = 1K) (1024 بایت)
- **آدرس منطقی** = شماره قطعه (Segment) = 1، افسست (تغییر) = 752 [0001001011110000] (نمایش باینری یک آدرس دیگر که برای segmentation مناسب است)
- **(ج) قطعه‌بندی (Segmentation)**

این نمودار سه روش مختلف برای تفسیر و تقسیم‌بندی یک آدرس حافظه مجازی (Logical Address) را مقایسه می‌کند:

1. تقسیم‌بندی (Partitioning) -

(الف): در این روش ساده، آدرس به عنوان یک عدد واحد و پیوسته در نظر گرفته می‌شود (مثلاً 1502). این روش مربوط به cXEM ‌های مدیریت حافظه ساده و اولیه است.

2. صفحه‌بندی (Paging) -

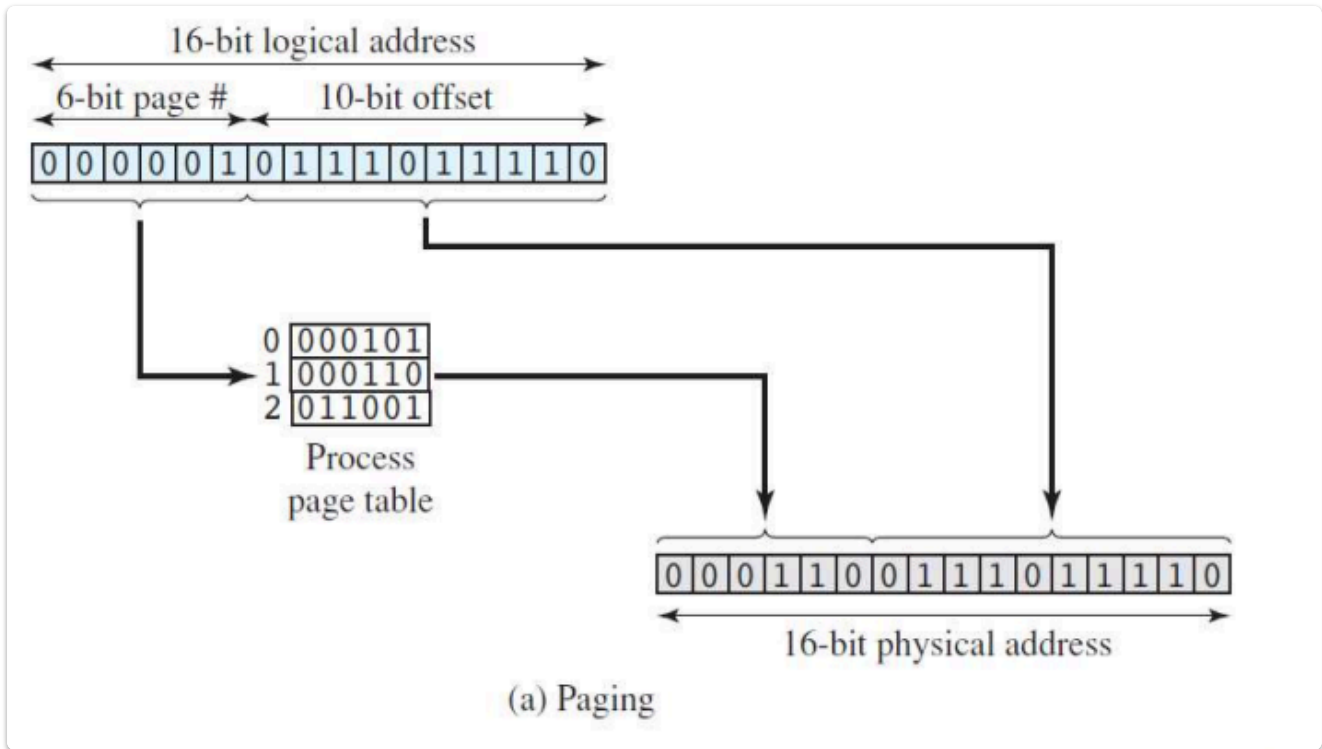
(ب): در این روش، آدرس منطقی به دو بخش تقسیم می‌شود: شماره صفحه (Page Number): بخشی از آدرس که مشخص می‌کند صفحه مورد نظر در کجای جدول صفحه قرار دارد (در این مثال: 1). افسست (Offset): بخشی از آدرس که مشخص می‌کند آدرس دقیق مورد نظر در داخل آن صفحه کجاست (در این مثال: 478). در این مثال 1K (1024 بایت) در نظر گرفته شده است. برای بدست آوردن شماره صفحه و افسست، آدرس کلی (1502) بر اندازه صفحه تقسیم می‌شود $1502 / 1024 = 1$ برای شماره صفحه و $1502 \% 1024 = 478$ برای افسست.

3. قطعه‌بندی (Segmentation) -

(ج): در این روش، آدرس منطقی نیز به دو بخش تقسیم می‌شود، اما بر اساس واحدهای منطقی برنامه: شماره قطعه (Segment Number): بخشی از آدرس که مشخص می‌کند کدام قطعه از برنامه (مثلاً کد، داده، پشته) ارجاع می‌دهد (در این مثال: 1). افسست (Offset): بخشی از آدرس که مشخص می‌کند آدرس دقیق مورد نظر در داخل آن قطعه کجاست (در این مثال: 752). بر خلاف صفحه‌بندی، اندازه قطعه‌ها معمولاً ثابت نیست و می‌تواند متغیر باشد.

نشان دادن این است که چگونه یک مقدار عددی یکسان (یا مشابه) می‌تواند بسته به طرح مدیریت حافظه مورد استفاده (Paging vs. Segmentation) به روش‌های مختلفی تفسیر و ترجمه شود تا به یک آدرس فیزیکی تبدیل گردد.

(Address Translation in Paging)



(این نمودار فرآیند تبدیل یک آدرس منطقی به آدرس فیزیکی را نشان می‌دهد)

آدرس منطقی ۱۶ بیتی ۶ بیت شماره صفحه - ۱۰ بیت افسست [`0 1 1 1 1 0 1 1 1 0 0 0 0`] (این نمایش باینری آدرس منطقی است که به دو بخش تقسیم شده است)

[`0 1 1 1 1 0 1 1 1`] (۶ بیت اول: شماره صفحه - ۱۰ بیت بعدی: افسست)

جدول صفحه فرآیند (Process page table) (این جدول شماره صفحه منطقی را به شماره قاب فیزیکی نگاشت می‌کند)

[`0 1 1 0 0 0`] [`0 1 1 1 1 0 1 1 1 1 0`] (خروجی جدول صفحه: شماره قاب فیزیکی جدید + افسست تغییرناپذیر)

آدرس فیزیکی ۱۶ بیتی [`0 1 1 1 1 0 1 1 1 1 0 0 1 1 0 0 0`] (نتیجه نهایی: آدرس فیزیکی در حافظه اصلی)

این نمودار فرآیند حیاتی ترجمه آدرس (Address Translation) را در سیستم مدیریت حافظه صفحه‌بندی (Paging) نشان می‌دهد.

مراحل تبدیل آدرس منطقی به فیزیکی:

۱. تقسیم آدرس منطقی (۱۶ بیتی):

واحد مدیریت حافظه (MMU) آدرس منطقی تولید شده توسط CPU را دریافت می‌کند.

این آدرس به دو بخش تقسیم می‌شود:

۶ بیت با ارزش (Most Significant Bits):

این بخش به عنوان شماره صفحه (Page Number) یا شاخص (Index) برای جستجو در جدول صفحه استفاده می‌شود. (در مثال: `000010` باینری که معادل عدد ۲ دهدهی است).

۱۰ بیت کم‌ارزش (Least Significant Bits):

این بخش افسست (Offset) نامیده می‌شود. این بخش مشخص می‌کند آدرس دقیق مورد نظر در داخل صفحه مورد نظر کجاست. از آنجایی که افسست ۱۰ بیت است، اندازه هر صفحه ۱K (`۱۰۲۴` بایت) است ($1024 = 10^3$). (در مثال: `1110111110`).

۲. جستجو در جدول صفحه (Process Page Table): شماره صفحه (مثلاً ۲) به عنوان اندیس برای دسترسی به جدول صفحه فرآیند استفاده می‌شود.

هر ورودی در این جدول حاوی شماره قاب (Frame Number) فیزیکی است که صفحه منطقی مورد نظر در آن قرار دارد. (در مثال: شماره قاب `000110` باینری که معادل عدد ۶ دهدهی است).

۳. ایجاد آدرس فیزیکی: شماره قاب فیزیکی به دست آمده از جدول صفحه، جایگزین شماره صفحه منطقی در آدرس می‌شود. افسست بدون تغییر باقی می‌ماند و مستقیماً به آدرس فیزیکی منتقل می‌شود. در نهایت، با ترکیب شماره قاب فیزیکی (۶ بیت) و افسست (۱۰ بیت)، آدرس فیزیکی ۱۶ بیتی نهایی ساخته می‌شود. (در مثال: $0001101110111110 = 1110111110 + 000110$).

نتیجه نهایی:

این فرآیند به CPU اجازه می‌دهد با استفاده از آدرس‌های منطقی (مجازی) کار کند، در حالی که سخت‌افزار به طور خودکار این آدرس‌ها را به مکان واقعی خود در حافظه فیزیکی ترجمه می‌کند. این اساس حافظه مجازی و محافظت از حافظه است.

Segmentation در Address Translation

در Segmentation، هر پردازش به Segments تقسیم می‌شود. Logical Address شامل:

- Segment Number (سمت چپ)
- Offset (سمت راست، موقعیت داخل)

برای دسترسی به حافظه، Logical Address باید به Physical Address ترجمه شود.

• مراحل ترجمه آدرس

استخراج Segment Number

تعداد n بیت سمت چپ آدرس منطقی → شماره Segment را مشخص می‌کند.

استفاده از Segment Number برای یافتن Base Address

Segment Number → پردازش Segment Table

شروع آدرس فیزیکی (Base Address) Segment بررسی Offset ها → Segment Table

Offset بیت سمت راست آدرس منطقی m=

سیستم عامل بررسی می‌کند که Offset > طول Segment باشد

اگر Offset ≤ طول Segment → آدرس Invalid (خطای حافظه)

• محاسبه Physical Address

Base Address = Physical Address (شروع Segment) + Offset

این آدرس فیزیکی است که برای دسترسی به RAM استفاده می‌شود

مثال ساده:

فرض کن یک پردازش سه Segment دارد:

Segment 0: Base = 1000, Length = 500

Segment 1: Base = 5000, Length = 1000

Segment 2: Base = 10000, Length = 200

Logical Address: Segment Number = 1, Offset = 200

Segment Table → Base Address = 5000

Offset = 200 < Length 1000 → Valid

Physical Address = 5000 + 200 = 5200

نکات کلیدی

Segmentation = Segment Number + Offset در Logical Address

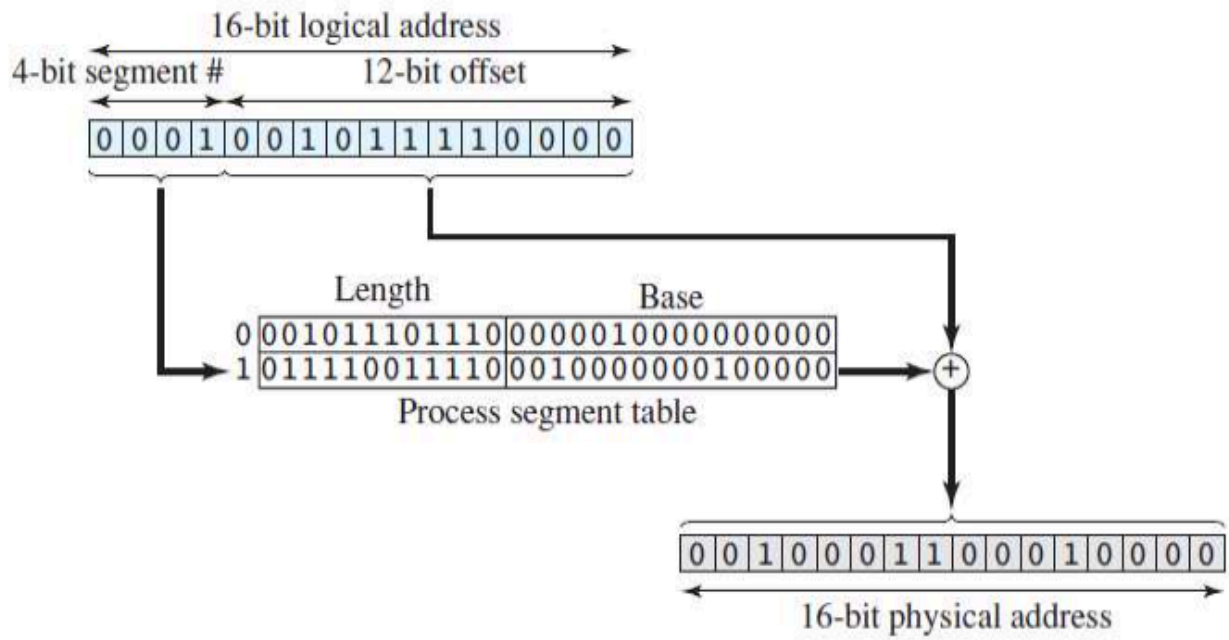
Length و Segment Table → Base Address

Physical Address = Base Address + Offset

بررسی Offset مهم است → جلوگیری از Invalid Memory Access



Address Translation in Segmentation



(b) Segmentation

End of Chapter 9 :)